

Formal Syntax and Semantics of the Interpreter

1 Context-Free Grammar (CFG)

The syntax of the language is defined by the following Context-Free Grammar. Non-terminals are capitalized, while terminals and operators are in lowercase or symbolic formats.

$$\begin{aligned} S &\rightarrow A \mid E \\ A &\rightarrow \text{VAR} = E \\ E &\rightarrow E \text{ bop } E \mid U \\ U &\rightarrow \text{uop } U \mid P \\ P &\rightarrow (E) \mid \text{VAR} \mid \text{VAL} \\ \text{VAL} &\rightarrow \text{NUM} \mid \text{TRUE} \mid \text{FALSE} \end{aligned}$$

The binary operators (bop) and unary operators (uop) are formally specified as the following terminal sets:

$$\begin{aligned} \text{bop} &\rightarrow + \mid - \mid * \mid / \quad (\text{Arithmetic}) \\ &\mid < \mid > \mid \leq \mid \geq \mid = \mid \neq \quad (\text{Relational}) \\ &\mid \&\& \mid \parallel \quad (\text{Logical}) \\ \text{uop} &\rightarrow - \mid ! \quad (\text{Arithmetic and Logical Negation}) \end{aligned}$$

The regex used to parse variables is: `'[A-Za-z_][A-Za-z_0-9]*$'` (Variables have to start with a letter or '_', then they can also contain numbers)

Parsing numbers is done using rust's `str.parse::< i32 >`

2 Semantics

2.1 Domains and Notations

To define the formal semantics of the language, we first establish the value domains, state notations, and operator representations.

- **Primary values:** $n \in \text{Int}$ (integers), $b \in \{\text{true}, \text{false}\}$ (boolean values).
- **Variables:** $x \in \text{Var}$.
- **Final value:** $v \in \text{Value} = \text{Int} \cup \text{Bool}$.
- **State (Environment):** $\sigma : \text{Var} \rightarrow \text{Value}$ represents a partial function mapping a variable name to its current value in memory.
- **Abstract Operators:** Let $\oplus \in \{+, -, *, /\}$ represent arithmetic operators, $\sim \in \{<, >, \leq, \geq, ==, \neq\}$ represent relational operators, and $\otimes \in \{\&\&, ||\}$ represent logical operators.

The notation $\langle E, \sigma \rangle \rightarrow v$ is read as: *Expression E evaluated in the context of state σ reduces to the value v .*

2.2 Axioms for Values and Variables

Literal values evaluate to themselves, regardless of the state.

$$\begin{aligned}\langle n, \sigma \rangle &\rightarrow n \\ \langle b, \sigma \rangle &\rightarrow b\end{aligned}$$

Variables extract their value directly from the state σ . If $\sigma(x)$ is undefined, the execution halts (undeclared variable error).

$$\frac{\sigma(x) = v}{\langle x, \sigma \rangle \rightarrow v}$$

2.3 State Modification

Let $\sigma[x \mapsto v]$ denote an updated environment that is identical to the current state σ , except that the identifier x is now bound to the value v .

The evaluation of an assignment requires first reducing the expression E to a final value v in the current state σ , and then producing the updated state:

$$\frac{\langle E, \sigma \rangle \rightarrow v}{\langle x = E, \sigma \rangle \rightarrow \sigma[x \mapsto v]}$$

2.4 Semantics of Arithmetic Operations

These rules enforce the type constraint: both operands must evaluate to values from the set Int.

$$\frac{\langle E_1, \sigma \rangle \rightarrow n_1 \quad \langle E_2, \sigma \rangle \rightarrow n_2}{\langle E_1 \oplus E_2, \sigma \rangle \rightarrow n_1 \oplus n_2}$$

Note: For the division operation ($/$), the additional condition $n_2 \neq 0$ is required.

2.5 Semantics of Relational Operations

These operations only accept values from Int as premises, but the resulting transition produces a value from Bool.

$$\frac{\langle E_1, \sigma \rangle \rightarrow n_1 \quad \langle E_2, \sigma \rangle \rightarrow n_2}{\langle E_1 \sim E_2, \sigma \rangle \rightarrow (n_1 \sim n_2)}$$

Note: The expression $(n_1 \sim n_2)$ on the right side of the transition arrow represents the mathematical evaluation of the proposition, yielding exactly true or false.

2.6 Semantics of Logical Operations

These operators require operands strictly evaluated to values from Bool. Let $\bullet \in \{\wedge, \vee\}$ be the mathematical equivalent of the abstract operator $\otimes$.

$$\frac{\langle E_1, \sigma \rangle \rightarrow b_1 \quad \langle E_2, \sigma \rangle \rightarrow b_2}{\langle E_1 \otimes E_2, \sigma \rangle \rightarrow b_1 \bullet b_2}$$

2.7 Semantics of Unary Operations

Arithmetic negation (Unary Minus): Requires a value of type Int.

$$\frac{\langle E, \sigma \rangle \rightarrow n}{\langle -E, \sigma \rangle \rightarrow -n}$$

Logical negation (NOT): Requires a value of type Bool.

$$\frac{\langle E, \sigma \rangle \rightarrow b}{\langle !E, \sigma \rangle \rightarrow \neg b}$$